

ACTUAL IMPLEMENTATION BASED DOCUMENTATION

Intelligent E-Commerce Price Comparison and Prediction Platform

A full-stack web application for searching products, comparing marketplace prices, tracking price history, receiving price alerts, and generating buy/wait recommendations using a Python ML microservice.

FRONTEND

React, TanStack Router/Start, Vite, Tailwind CSS, React Query, Recharts

BACKEND

Node.js, Express.js, MongoDB, Mongoose, JWT authentication

ML SERVICE

Flask, scikit-learn, pandas, NumPy, joblib, Playwright-backed Flipkart scraping

PREPARED ON

7 May 2026

This report has been corrected to match the implemented project codebase and removes unsupported or unused claims from earlier reference material.

Table of Contents

01	Abstract	03
02	Introduction and Problem Statement	04
03	Objectives and Scope	05
04	Technology Stack	06
05	System Architecture	07
06	Module Implementation	08
07	Database Design	12
08	API Design	14
09	Machine Learning Implementation	16
10	Testing, Limitations, and Future Scope	18
11	Conclusion	20

Abstract

01

PriceLens is an intelligent e-commerce price comparison platform developed as a three-service application. The frontend provides a modern product discovery interface, search results, product detail pages, analytics, price alerts, a dashboard, admin views, and community discussion pages. The backend exposes a REST API for authentication, product search, comparison, alerts, admin analytics, forum posts, and communication with the ML service. The ML service provides price prediction and recommendation endpoints and also serves as the bridge for Flipkart scraping.

The implemented system aggregates product data from Amazon through RapidAPI, eBay through the official Browse API, Walmart through RapidAPI, and Flipkart through a Python scraping endpoint. Products are normalized into a shared format, stored in MongoDB, and enriched with price history records. The platform then uses these records to show comparisons, charts, and machine-learning-assisted recommendations.

Corrected scope: the implementation does not monitor hundreds of marketplaces. The actual marketplace integrations present in code are Amazon, eBay, Walmart, Flipkart, and fallback mock/scraped data for unavailable external results.

3

Application services: client, server, and ML service.

4

Main marketplace paths: Amazon, eBay, Walmart, and Flipkart.

MongoDB

Stores users, products, alerts, price history, forum data, and search logs.

Introduction

02

Online shoppers often compare prices manually across multiple websites before making a purchase. This process is time-consuming and can still miss better deals, historical price patterns, or future price drops. PriceLens addresses this problem by combining product aggregation, price comparison, alerting, historical tracking, and prediction in one application.

Problem Statement

The project aims to design and implement a full-stack platform where users can search for a product, compare its prices across available marketplaces, inspect price history, and receive actionable recommendations about whether to buy now or wait.

Need for the System

- Prices vary across platforms and can change frequently.
- Manual comparison does not preserve historical pricing trends.
- Users need alerts when a product reaches a target price.
- Decision support is more useful when historical data and prediction are combined.

Implemented Outcome

The delivered project includes a React frontend, Express backend, MongoDB database layer, and Flask ML microservice. It supports product search, marketplace comparison, product detail pages, watchlist behavior, price alert creation, admin statistics, forum discussions, and ML-based recommendation fallbacks.

Objectives and Scope

03

Objectives

- Build a responsive frontend for searching and viewing products.
- Fetch product results from supported external marketplace services.
- Normalize different marketplace response formats into a single product model.
- Persist product and price history data in MongoDB.
- Provide comparison, history, prediction, and redirect APIs.
- Support user accounts, refresh-token authentication, dashboards, watchlists, and alerts.
- Expose admin and community forum modules for platform management and user discussions.

Implemented Scope

Area	Implemented in Code
Product Search	Search endpoint aggregates Amazon, eBay, Flipkart, Walmart, and fallback products.
Price Comparison	Product sources are embedded under each Product document and sorted by price.
Price History	PriceHistory documents are inserted during product upsert for valid source prices.
ML Prediction	Express calls Flask /predict and /recommend through mlBridgeService with retry and fallback.
Alerts	JWT-protected alert CRUD routes with target price, target percentage, cooldown, and email support.
Community	Forum post and comment APIs with voting and protected editing/deletion.
Admin	Admin dashboard route and protected API for statistics, users, products, and alerts.

Out of Scope or Not Fully Implemented

- No evidence of live monitoring across 200+ stores.
- Some frontend dashboard charts use placeholder chart data where backend time-series analytics are not present.

- Homepage product cards use local mock products; product detail handles them separately from real MongoDB products.
- Mock product purchase links are converted to store search URLs, not exact product listing URLs.

Technology Stack

Frontend

React 19, TanStack Router/Start, Vite, TypeScript, Tailwind CSS, React Query, Zustand, Axios, Recharts, Framer Motion, Lucide React, and reusable UI components.

Backend

Node.js, Express.js, Mongoose, MongoDB, JWT, bcryptjs, express-validator, express-rate-limit, Helmet, CORS, Winston logging, Morgan, Nodemailer, and node-cron.

ML Service

Python Flask, flask-cors, scikit-learn, pandas, NumPy, SciPy, joblib, BeautifulSoup, lxml, Playwright, Playwright stealth, requests, and dotenv.

Database and Tools

MongoDB stores application data. Jest, Supertest, and mongodb-memory-server are available for backend tests. Flask tests exist under the ML service.

Runtime Ports

Service	Port	Purpose
Client	5173	Vite/TanStack frontend application.
Server	5000	Express REST API mounted under /api.
ML Service	8000	Flask endpoints for prediction, recommendation, reports, and Flipkart scraping.

System Architecture

05

PriceLens follows a modular architecture where the frontend communicates with the Express backend, the backend persists data in MongoDB, and ML-related requests are delegated to the Flask microservice. External product integrations are isolated in service files and normalized before being stored.

React Frontend
TanStack routes

Express API
/api routes

Service Layer
marketplace + ML
bridge

MongoDB
Mongoose models

Flask ML Service
prediction +
scraper

Client requests -> API validation -> service aggregation -> storage and response -> charts/recommendations

High-Level Request Flow

1. User searches for a product from the frontend search page.
2. The frontend calls `GET /api/products/search?q=...`.
3. The backend queries Amazon, eBay, Flipkart, and Walmart services where available.
4. Raw results are normalized into a common product format.
5. Products are upserted into MongoDB and price history points are inserted.
6. The frontend receives unified product responses and renders comparison cards.
7. Product detail pages call prediction and history APIs for deeper decision support.

Service Boundaries

Layer	Responsibility
Frontend	UI rendering, routing, state, forms, charts, user actions, and API calls.
Backend API	Validation, auth, aggregation, database operations, cache, rate limiting, and error handling.
ML Service	Prediction, buy/wait/watch recommendation, model report endpoint, and Flipkart scraping.
Database	Persistent product, price, alert, user, forum, comment, and search records.

Frontend Implementation

The frontend is built as a Vite/TanStack application. It uses route-based pages, shared components, Axios API helpers, and React Query for server-state fetching. Product cards are reused across homepage, trending, and search views.

Main Pages

Route	Implemented Function
/	Homepage with search, feature sections, and local trending product cards.
/search	Product search with filters, sorting, pagination, grid/list layouts, and ProductCard display.
/product/\$id	Product detail, offers, ML recommendation, chart, alerts, and Amazon reviews when backend product has ASIN.
/trending	Fetches backend trending products and adapts them to frontend product cards.
/dashboard	Authenticated user dashboard using watchlist and alert data.
/alerts	User price alert list and delete functionality.
/forum	Community post listing, filtering, sorting, and post creation modal.
/admin	Admin-only dashboard with overview, user management, products, and analytics UI.

Important Frontend Corrections

- Homepage products use local slugs, not MongoDB ObjectIds. The product detail route now detects this and loads matching mock products locally.
- Mock product offer links originally used `#`. The detail page now generates real store search URLs when a direct product URL is unavailable.
- Backend product IDs still use the original API path and MongoDB validation.

State and Data Handling

Public product APIs are wrapped in `productApi.ts`. Authenticated actions use `axiosPrivate`, which attaches access tokens and refreshes tokens on 401 responses. Product responses from

backend are transformed through `adaptProduct` so frontend components can use a consistent shape

Frontend Implementation

08

Backend Implementation

06B

The backend is an Express REST API. It applies Helmet, CORS, JSON parsing, global rate limiting, request logging, route-level validation, centralized error handling, and MongoDB persistence through Mongoose.

Controller Responsibilities

Controller	Responsibility
authController	Registration, login, refresh token, profile, logout.
productController	Search, upsert, product details, history, compare, prediction, redirect, trending, autocomplete, watchlist, reviews.
alertController	Create, list, update, pause/resume, delete, and test alerts.
adminController	Dashboard stats, users, products, alerts, and admin management operations.
forumController	Forum posts, comments, votes, user posts, and protected edits.

Product Search Pipeline

1. Validate query parameters using `express-validator`.
2. Build or read a cache key for repeated searches.
3. Fetch marketplace data in parallel with `Promise.allSettled`.
4. Normalize Amazon, eBay, Flipkart, and Walmart results.
5. Merge products by normalized title.
6. Upsert products and insert price history entries.
7. Apply category, price, and platform filters.
8. Sort and return a unified response.

Security and Reliability

- JWT authentication protects private routes such as alerts, dashboard data, watchlist, and admin APIs.
- Passwords are hashed with `bcrypt` before saving.

- Helmet, CORS, validation, and rate limiting are configured in the Express app.
- ML calls use retry behavior, caching, and fallbacks when the Flask service is unavailable.

Integration Modules

06C

Marketplace Integrations

Marketplace	Implementation	Notes
Amazon	RapidAPI Real-Time Amazon Data API	Search, product details by ASIN, category list, and reviews.
eBay	Official eBay Browse API	Uses OAuth2 application token and marketplace ID configuration.
Walmart	RapidAPI realtime Walmart data	Search and rollback/deals endpoints.
Flipkart	Backend calls Flask scraper endpoint	Python service uses scraping logic and returns normalized raw data.
Fallback	Scraper service fallback products	Used only when live integrations return no products.

Normalization

The `productNormalizer` service converts platform-specific responses into one product shape with fields for title, brand, image, category, description, ASIN, search keywords, and source-specific price data. Currency conversion normalizes source prices into the configured default currency, which defaults to INR.

ML Bridge

The Express backend does not directly run model inference. Instead, it sends product history and metadata to the Flask service through `mlBridgeService`. This bridge adds retries, timeout handling, cache support, payload normalization, and graceful fallback responses.

Implementation correction: eBay uses the Browse API rather than scraping. Flipkart is the marketplace connected through the Python scraping endpoint.

Feature Modules

06D

User Authentication

Users can register, log in, refresh tokens, fetch their profile, update profile data, and log out. Private frontend requests use access tokens stored in local storage and automatically request refresh tokens when a private request returns 401.

Price Alerts

Alerts are protected by JWT authentication. Each alert stores `userId`, `productId`, `targetPrice`, optional percentage drop target, `baselinePrice`, `platform`, `active` status, `triggered` state, notification flags, `cooldown` hours, `recurring` behavior, and notes.

Watchlist and Dashboard

Users can add or remove products from their watchlist through the product watchlist endpoint. The dashboard fetches watchlist product data and user alerts to show tracked products, basic savings, alert counts, and category summaries.

Admin Module

Admin UI guards access by checking user role. The admin area contains overview statistics, user management, and product/alert views. Some chart series in the UI are presentation placeholders because backend time-series analytics are not fully implemented.

Forum Module

Forum APIs allow listing posts publicly, reading individual posts, filtering by tag and query, creating posts with authentication, editing or deleting owned posts, adding comments, editing comments, deleting comments, and voting on posts or comments.

Database Design

07

MongoDB is used as the persistent database, with Mongoose schemas defining the application data model. The design centers on product aggregation and user-specific tracking.

Main Collections

Collection	Purpose	Key Fields
User	Accounts and preferences.	name, email, password, role, watchlist, refreshToken, notificationPrefs.
Product	Aggregated product records.	title, normalizedTitle, brand, category, imageUrl, sources, lowestPrice, highestPrice, averagePrice, asin.
PriceHistory	Historical price points.	productId, platform, price, originalPrice, currency, availability, timestamp.
Alert	User price alerts.	userId, productId, targetPrice, targetPctDrop, platform, triggered, cooldownHours.
SearchHistory	Searches and clicked products.	userId, query, resultCount, clickedProductId, source, ipAddress.
ForumPost	Community posts.	title, content, tags, userId, productId, votes.
Comment	Forum comments.	postId, userId, text, parentId, votes.

Product Source Subdocument

Each Product contains an embedded `sources` array. Every source stores platform, price, currency, original price, discount, link, availability, rating, review count, seller, and last updated time. This allows one product record to hold marketplace-specific current prices without requiring a separate collection for current source snapshots.

Indexes

- Product text index over title, searchKeywords, and brand.
- Product indexes over lowestPrice, category plus lowestPrice, and searchCount.
- Alert indexes for active alert cron checks, user alert dashboard queries, and product-level alert lookups.

- User indexes for role and active status plus recent creation sorting.

API Design

08

Product APIs

Method	Endpoint	Actual Purpose
GET	/api/products/search	Search and aggregate products.
GET	/api/products/trending	Fetch Walmart rollbacks and DB trending products.
GET	/api/products/autocomplete	Return suggestions from search history and product text search.
GET	/api/products/:id	Fetch MongoDB product by ObjectId.
GET	/api/products/:id/history	Fetch grouped and flat price history.
GET	/api/products/:id/compare	Return source comparison sorted by price.
GET	/api/products/:id/predict	Fetch ML prediction and recommendation.
GET	/api/products/:id/redirect	Return selected platform purchase URL.
GET	/api/products/:id/amazon-reviews	Fetch live Amazon reviews only when ASIN is available.
POST	/api/products/:id/watchlist	Toggle authenticated user's watchlist item.

Other Main APIs

Area	Endpoint Pattern	Notes
Auth	/api/auth/*	Register, login, refresh, me, profile, logout.
Alerts	/api/alerts/*	Protected alert CRUD and test email endpoint.
Forum	/api/forum/*	Public reads and protected write/vote actions.
Admin	/api/admin/*	Role-protected management APIs.
ML Proxy	/api/ml/*	Backend bridge routes to ML status/report behavior.

Validation

Search query, pagination, platform, sorting, category, price ranges, product ObjectIds, history days, and autocomplete query values are validated by express-validator. Invalid product IDs return a 422 validation response. This is why homepage mock product slugs are handled locally by the frontend detail page rather than sent to the product API.

Machine Learning Implementation

09

The ML service is implemented as a separate Flask application. It loads serialized model artifacts from the `models` directory when available and exposes prediction, recommendation, batch prediction, model report, health, and Flipkart scrape endpoints.

Prediction Logic

1. Receive product price history and optional metadata from Express.
2. Require at least three historical price points for prediction.
3. Build date, lag, rolling window, momentum, category, platform, rating, review count, and MRP features.
4. Use trained pipeline artifacts when present.
5. Blend primary model output with linear regression when that artifact is available.
6. Fallback to linear extrapolation when trained artifacts are unavailable.
7. Return predicted price, confidence, price range, method, and current price.

Recommendation Logic

A deterministic rule-based recommendation converts predicted movement into `BUY_NOW`, `WAIT`, or `WATCH`. A falling trend with meaningful savings recommends waiting. A rising or stable trend tends toward buying now. Unknown or insufficient data produces watch-style responses.

ML Endpoints

Endpoint	Purpose
<code>GET /health</code>	Returns service status and model readiness.
<code>POST /predict</code>	Predicts future price for one product.
<code>POST /recommend</code>	Generates a buy/wait/watch signal.
<code>POST /batch-predict</code>	Predicts multiple products in one request.
<code>GET /model/report</code>	Returns training evaluation report if present.

Testing and Verification

10A

Available Automated Tests

Service	Testing Tools	Coverage Present
Backend	Jest, Supertest, mongodb-memory-server	Auth and product search endpoint tests are present.
ML Service	Python tests	ML service test file exists under <code>ml-service/tests</code> .
Frontend	Build verification	Vite production build validates route/type/bundling behavior.

Recent Verification During Report Preparation

- The frontend production build completed successfully after the product detail fixes.
- The homepage mock-product validation issue was resolved by bypassing backend ObjectId validation for local slug products.
- The homepage mock-product purchase link issue was resolved by generating external marketplace search URLs when offer links are placeholders.

Suggested Test Cases

- Search returns normalized products from mocked marketplace services.
- Invalid product IDs return validation errors from backend APIs.
- Homepage mock products open detail pages without backend validation calls.
- Mock product buy links open correct marketplace search URLs.
- JWT-protected alert routes reject unauthenticated users.
- ML prediction returns fallback when price history has fewer than three points.

Limitations and Future Scope

10B

Current Limitations

- External marketplace coverage is limited to the integrations present in code.
- External APIs require valid credentials and may return empty results when keys are missing or rate limits are reached.
- ML accuracy depends on sufficient historical price records and available trained artifacts.
- Homepage products are demo data, not database records.
- Some admin and dashboard visualizations are UI placeholders rather than fully backend-driven time-series analytics.
- Mock product purchase URLs search marketplaces instead of opening exact listings.

Future Enhancements

- Add more official marketplace APIs with stable product identifiers.
- Replace homepage demo products with backend-driven trending products.
- Add full frontend automated tests with component and route coverage.
- Add backend time-series analytics for admin activity charts.
- Improve model training pipeline documentation and automate periodic retraining.
- Add a browser-safe redirect tracking endpoint for all external purchase clicks.
- Improve alert UX for percentage-drop alerts and recurring alert controls.

Deployment Considerations

Production deployment should define environment variables for MongoDB, JWT secrets, API keys, email credentials, client origin, and ML service URL. The frontend should point `VITE_API_URL` to the deployed backend. Backend and ML service logs should be monitored because marketplace integrations depend on external services.

Conclusion

11

PriceLens successfully implements a practical full-stack product comparison and price intelligence platform. It combines a modern React frontend, Express API backend, MongoDB persistence layer, and Python Flask ML service into a working architecture. The project supports product search, marketplace comparison, product detail pages, price history, alerts, dashboards, admin management, community discussions, and recommendation logic.

The corrected implementation shows that the strongest part of the project is its modular service design: marketplace adapters, product normalization, MongoDB persistence, and ML bridging are separated cleanly enough to extend later. The system is not a universal marketplace monitor, but it is a functional and extensible foundation for intelligent e-commerce comparison.

References

- Project source code under `client/`, `server/`, and `ml-service/`.
- React, TanStack Router, Vite, Tailwind CSS, Express.js, MongoDB, Mongoose, Flask, and scikit-learn documentation.
- RapidAPI marketplace data services and eBay Browse API integration code used in this project.

This report intentionally avoids unsupported statements such as "200+ stores", "fully real-time monitoring for all products", or "exact Amazon redirect for demo products", because those statements are not supported by the current implementation.